ZAMZAM UNIVERSITY OF SCIENCE & TECHNOLOGY SOMALIA

FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & INFORMATION SYSTEMS

CARUURKAB AI: AN ARTIFICIAL INTELLIGENCE-BASED LEARNING PLATFORM FOR SOMALI CHILDREN

A project submitted in Partial fulfill of the requirement for the award of the Bachelor Degree of Computer Science & Information Systems

Prepared By:

1. CIS:220420 - Mohamed Ali Mohamed
2. CIS:220138 - Abdirahman Ali Abdulle
3. CIS:220448 - Mohamed Abdi Mohamud
4. CIS:230382 - Adan Ibrahim Ahmed
5. CIS:220745 - Mohamed Abdukadir Abdi
6. CIS:220303 - Jamac Mohamed Abdinor

Supervisor:

Prof. Ahmed Geedi
Lecturer & Academic Supervisor
Faculty of Computing & IT

Academic Year: 2025–2026

Statement of Original Authorship

The work contained in this thesis has not been previously submitted to meet requirements for an award at this or any other higher education institution. To the best of my knowledge and belief, the thesis contains no material previously published or written by another person except where due reference is made. We hereby declare that this final year project report entitled "Caruurkab AI: An Artificial Intelligence-Based Learning Platform for Somali Children" is our original work, conducted under the academic supervision of Prof. Ahmed Geedi. All primary software architectures, database designs, implementation details, and experimental evaluations described herein represent the independent efforts of the project team.

Prepared By:

_____

The Project Team

Approved By:

_____

Supervisor: Prof. Ahmed Geedi

DEDICATION

To our beloved parents, whose unconditional love, sacrifices, prayers, and continuous support have been our constant source of strength and inspiration throughout our academic journeys.

To our esteemed lecturers and advisors at Zamzam University, who dedicated their time, knowledge, and expertise to shape our skills and prepare us to address real-world challenges through technology.

To the future children of Somalia, who deserve access to the highest quality of education and modern technological innovations to help them grow and rebuild our nation.

ACKNOWLEDGEMENT

First and foremost, we express our profound gratitude to Almighty Allah for His infinite blessings, strength, patience, and guidance, which enabled us to complete this final year project successfully.

We extend our deepest gratitude and respect to our project supervisor, Prof. Ahmed Geedi, for his invaluable guidance, constructive critiques, academic mentorship, and constant encouragement throughout the design, development, and reporting phases of this project. His high standards of academic excellence and expertise in software engineering significantly shaped the direction and outcomes of this research.

We are also highly grateful to the Dean and all faculty members of the Faculty of Computing and Information Technology at Zamzam University for providing us with a supportive academic environment, modern laboratory facilities, and resources that made the development of this project possible.

Lastly, we offer our heartfelt appreciation to our families and friends for their constant prayers, understanding, and encouragement during the late nights of coding and document compilation. Their support was vital to our success.

ABSTRACT

The integration of Artificial Intelligence (AI) in early childhood education has opened new pathways for personalized, adaptive learning. However, Somali-speaking children face a significant barrier due to the severe lack of localized digital learning resources. Most educational mobile applications are designed in English or Arabic, restricting access for early learners who have not yet mastered foreign languages. This project presents the design, development, and evaluation of Caruurkab AI, an intelligent, adaptive educational mobile application specifically designed for Somali-speaking children aged 3 to 8 years.

The application is built on a modern, robust architecture. The frontend is developed using Google's Flutter framework to ensure cross-platform compatibility, smooth animations, and high performance on budget Android devices. The backend is powered by Firebase for secure user authentication, push notifications, and media storage, alongside Supabase as a relational PostgreSQL database to manage complex progress tracking and school curriculum structures. An AI engine is integrated to analyze child performance and dynamically recommend personalized learning paths (Talo AI), adjust quiz difficulty, and offer voice-interactive guidance and automated conversational help through a student-facing chatbot (Wadahadal).

Caruurkab AI targets four main subject areas: Af Soomaali (Somali language), English (basic literacy), Mathematics (numeracy and basic arithmetic), and Saynis (introductory natural science). The system incorporates a comprehensive Admin Command Center that enables course content orchestration (Content Studio), quiz database updates (Su'aalo Direct), and real-time usage analytics reporting. Empirical evaluation of the system conducted with teachers and children in Mogadishu indicates a significant improvement in student engagement (84% increase), knowledge retention, and administrative efficiency. The results demonstrate that localized AI platforms are highly effective in transforming passive device consumption into productive, scaffolded learning experiences.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

In the contemporary era of the Fourth Industrial Revolution, technology has become an inseparable part of human life. Among all emerging technological innovations, Artificial Intelligence (AI) stands out as the most transformative force, refining workflows, industrial processes, communication, and learning methodologies globally. Educational Technology (EdTech) has progressed from simple digitizations of physical textbooks to highly interactive, smart systems capable of understanding, predicting, and adapting to individual student needs. This rapid progression is reshaping how knowledge is structured and absorbed.

Early childhood education (ECE) is universally recognized by neuroscientists and educational psychologists as the most critical phase of human cognitive development. During the developmental window of 3 to 8 years, the human brain develops neural pathways at an unprecedented speed. Cognitive capabilities, linguistic skills, logical reasoning, and basic behavioral patterns are established during this period. Educational research demonstrates that children learn concepts faster and with much higher long-term retention when they are actively engaged through interactive visuals, positive reinforcements, and, most importantly, instruction in their native language (mother tongue). Teaching children in foreign languages before they master their mother tongue limits cognitive growth.

Despite these global advancements, Somalia faces unique and severe educational challenges. Decades of socio-political instability have impacted the formal school system, leaving a lack of physical schools, trained early childhood educators, and standardized learning materials. A large percentage of Somali children do not have access to formal early childhood education. While physical classrooms are limited, mobile internet access and smartphone penetration have increased rapidly in Somalia. Telecommunication networks provide high-speed internet across major cities, and smartphones have become common items in Somali households. Consequently, there is a major opportunity to deliver educational resources directly to homes through mobile platforms.

However, this digital expansion has not translated into positive educational opportunities for Somali children. There is a complete lack of educational software designed specifically for the Somali language and local cultural context. Somali children who get access to smartphones spend their screen time passively watching foreign-language cartoon videos or playing interactive mobile games that offer no educational benefits. The educational apps available on global app stores are built in English, Arabic, or French. Young Somali children cannot use these applications independently because they do not speak these foreign languages at that age, which often leads to frustration, visual fatigue, and disengagement from productive learning activities.

This project introduces Caruurkab AI, an AI-powered educational mobile platform built specifically to address these challenges. Caruurkab AI combines the cross-platform capabilities of Google's Flutter

framework with the relational data management of Supabase and cloud infrastructure of Firebase. The application uses AI algorithms to build personalized learning paths, track progress in real-time, and offer voice assistance in the Somali language. By digitizing basic curriculum components (Af Soomaali, English, Math, and Saynis) into interactive modules, Caruurkab AI transforms standard mobile devices into intelligent learning assistants that help young children learn independently and establish strong cognitive foundations.

By utilizing modern software engineering paradigms and integrating localized artificial intelligence, the platform establishes a scalable model for digital ECE in developing countries. Somali families, particularly those in underserved areas without access to formal schools, can utilize the app to scaffold their child's intellectual development. The focus is to transform a child's interaction with mobile screens from passive consumption of foreign cartoons into an active, localized, and intelligent learning experience.

Furthermore, the implementation of localized educational software plays a significant role in establishing cognitive equity. When children learn in a language they speak fluently, they are able to understand complex logical structures without experiencing linguistic confusion. This accelerates their logical reasoning, spatial awareness, and vocabulary absorption.

In the context of Somalia, the lack of educational resources is compounded by historical socio-economic factors. Decades of civil conflict dismantled the formal school infrastructure, leaving the responsibility of early childhood education primarily in the hands of parents and private institutions. Mobile technology presents a scalable solution to bypass physical barriers and deliver structured, quality learning directly to the home environment.

From a technological perspective, the deployment of cross-platform frameworks like Flutter enables a single codebase to support multiple OS versions. This drastically reduces development costs and maintenance overheads, which is a key factor for sustainability in low-resource environments. The combination of lightweight frontend frameworks with serverless cloud architectures marks a significant transition in educational resource distribution in East Africa.

1.2 Problem Statements

The current state of early childhood education in Somalia is characterized by a mismatch between technological access and educational content. While smartphones and high-speed mobile internet are widely available in Somali households, children lack access to educational tools designed for their language and culture. This lack of localized software creates several critical problems:

First, young Somali children (ages 3 to 8) cannot use global educational apps independently because these platforms are built in foreign languages like English or Arabic. When children interact with these apps, they struggle with language barriers instead of learning the educational concepts, which often requires constant external supervision and translation. This prevents independent learning and places a heavy burden on parents who may not have the time or language skills to assist. The lack of interactive materials in the

mother tongue leaves early learners struggling to master basic literacy and numeracy concepts, slowing down overall cognitive development.

Second, because of this lack of localized educational tools, children spend their screen time on passive entertainment, such as unsupervised online videos and mobile games. This unstructured screen time contributes to screen addiction, lack of focus, and exposure to culturally irrelevant content, while offering no educational progress or intellectual stimulation. Passive video consumption decreases active attention span and hinders cognitive development during the child's most critical developmental stage.

Third, school administrators and content creators lack modern tools to distribute, update, and monitor localized educational curriculum. Without a centralized system, updating lessons and quiz questions is extremely difficult, and tracking overall system usage is impossible. Traditional administrative dashboards do not provide analytical summaries of student progress, making curriculum adjustment hard. There is a clear gap in tools that allow local educators to easily orchestrate and adjust early childhood learning materials.

Fourth, the Somali educational ecosystem lacks integration of modern technology, specifically Artificial Intelligence. Conventional educational materials follow a one-size-fits-all model, ignoring the fact that children learn at different speeds. Without adaptive learning systems, slow learners get left behind while fast learners become disengaged. Caruurkab AI is designed to address this problem by providing a localized, AI-driven mobile app that delivers early childhood curriculum in Somali with adaptive pathing.

Additionally, the absence of standardized early childhood software in the Somali language has resulted in a digital divide. While children in developed countries interact with smart, adaptive systems that scaffold their math and language acquisition, Somali children are restricted to passive consumer roles. This passive consumption does not build logical reasoning, motor coordination, or interactive cognitive processing.

Furthermore, traditional curriculum orchestration models are slow and paper-based. Teachers must manually update lesson guidelines and print quiz sheets, which is expensive and time-consuming. Without centralized database tables, it is impossible to audit whether the curriculum is effective, which student demographics struggle with specific subjects, or how quiz scores evolve over time. The absence of a centralized, secure admin content console prevents local educators from modernizing ECE curricula.

1.3 Research Questions

To guide the design, development, and evaluation of Caruurkab AI, this study addresses the following research questions:

1. How can we design a user interface that allows young Somali children (ages 3 to 8) to navigate and learn independently without constant adult supervision?
2. In what ways can Artificial Intelligence algorithms be used to personalize early childhood education and accommodate different learning speeds in a localized mobile app?

3. How can we structure a relational database using Supabase to manage complex progress tracking, curriculum mapping, and student performance history?
4. How effective is an admin-facing Command Center in helping educators edit lessons, update quiz databases, and monitor student metrics?
5. What is the impact of Caruurkab AI on child engagement, focus, and knowledge retention compared to traditional non-adaptive learning materials?

These research questions serve as the framework for the entire research process, directly informing the development sprints, the database schema design, and the final User Acceptance Testing (UAT) questionnaire. Each question addresses a key architectural, pedagogical, or administrative challenge in localized EdTech.

1.4 Objectives of the Study

General Objective:

The primary objective of this project is to design, develop, and evaluate an Artificial Intelligence-powered mobile learning platform (Caruurkab AI) specifically tailored for Somali-speaking children to provide personalized, engaging, and localized education.

Specific Objectives:

1. To design and implement a child-friendly, visually engaging user interface optimized for early learners aged 3 to 8 years.
2. To develop localized educational content in the Somali language covering basic literacy, numeracy, English, and introductory science.
3. To integrate an AI recommendation engine that dynamically adjusts lesson progression (Talo AI) and quiz difficulty based on individual performance.
4. To implement a student-facing chatbot (Wadahadal) for immediate voice-based guidance and response help.
5. To develop a comprehensive Admin Command Center that enables administrators to manage course content (Content Studio), update questions (Su'aalo Direct), and view user reports.
6. To build a secure and scalable backend system using Supabase (for relational PostgreSQL database management) and Firebase (for authentication, media hosting, and notifications).
7. To evaluate the usability, effectiveness, and user acceptance of Caruurkab AI among children, educators, and administrators in Mogadishu.

The specific objectives are structured logically to guide the development process. Objectives 1 and 2 focus on user interface and localized content, which are the baseline requirements for usability. Objectives 3 and 4 address the AI-driven recommendation and chatbot, which are the core intelligent features. Objectives 5 and 6 handle backend relational database setups and admin consoles, ensuring system scalability and security.

Objective 7 validates the entire platform through empirical testing in Mogadishu.

1.5 Scope of the Study

The scope of this project is defined by geographical, age-group, curriculum, and technological boundaries:

Geographically, the study and user testing were conducted in Mogadishu, Somalia, involving local early childhood education schools. The target user group is Somali-speaking children aged 3 to 8 years, along with school administrators and early childhood educators.

In terms of curriculum, the app covers four basic subjects: Af Soomaali (alphabet, vocabulary, basic grammar), English (alphabet, phonics, basic words), Mathematics (numbers, counting, basic addition and subtraction), and Saynis (animals, plants, human body parts, seasons). The lessons are structured based on the standard Somali primary school curriculum guidelines.

Technically, the project is delimited to a cross-platform mobile application built using the Flutter framework and Dart language, optimized for Android mobile devices. The backend utilizes Firebase for user authentication and media file hosting, and Supabase (PostgreSQL) for transactional database operations and user progress tracking. The application requires an active internet connection for database synchronization, media streaming, and AI chatbot responses, though basic offline caching is implemented for offline lesson access.

The technical scope is defined to ensure system stability. Flutter's rendering engine compiles directly to native ARM code, which ensures smooth animations even on low-end processors. Firebase Auth and Supabase PostgreSQL RLS rules provide security boundaries, ensuring that data queries are checked for authentication at the database level before execution.

1.6 Research Hypotheses

To evaluate the educational effectiveness and engagement impact of the Caruurkab AI platform, the following hypotheses were formulated:

Hypothesis 1:

Null Hypothesis (H0): The integration of personalized AI recommendations and localized mother tongue chatbot guidance in Caruurkab AI has no significant effect on student lesson engagement and quiz passing rates compared to traditional non-adaptive digital learning tools.

Alternative Hypothesis (H1): The integration of personalized AI recommendations and localized mother tongue chatbot guidance in Caruurkab AI significantly increases student lesson engagement (measured by session duration and active interactions) and quiz passing rates by at least 25%.

Hypothesis 2:

Null Hypothesis (H0): There is no significant difference in the administrative efficiency and content orchestration time for school educators using the Admin Command Center compared to traditional manual database updates.

Alternative Hypothesis (H1): The use of the Admin Command Center significantly improves administrative efficiency, reducing the time required to update curriculum lessons and manage quiz questions by at least 50%.

The formulation of these hypotheses allows the research team to apply empirical statistical methods to evaluate the system. The alternative hypotheses (H1) are based on the premise that interactive gamification combined with localized AI chatbot assistance will keep early learners engaged longer than passive text viewers, leading to higher concept retention and quiz passing rates.

Similarly, the administrative hypothesis assumes that providing a visual, forms-based Content Studio and Su'aalo Direct interface eliminates the need for manual SQL script execution, reducing the administrative update timeline.

1.7 Significance of the Study

This research project holds significant value for multiple stakeholders within the Somali educational and technological ecosystem:

For Young Children: It provides an interactive, culturally relevant learning environment in their native language. It helps them build basic literacy, numeracy, and cognitive skills independently through gamified lessons that adapt to their needs, transforming screen time into productive, active cognitive learning.

For Educators & Administrators: It offers a centralized Admin Command Center to manage localized courses, edit quiz banks in real-time, and monitor student enrollment and metrics without manual paperwork or complex database operations.

For Local Schools: It demonstrates how digital technologies and AI tools can support early childhood education. It serves as a model for integrating mobile learning tools into primary schools and home study environments, showcasing modern EdTech possibilities.

For Software Developers: It provides design guidelines, architectural patterns, and technical templates for building localized, low-resource language educational apps, showing how Flutter, Firebase, and Supabase can be combined effectively.

Moreover, the study provides valuable empirical evidence regarding child-device interaction in developing nations. By documenting how Somali children navigate touchscreen interfaces, select subjects, and react to audio narations, the study establishes design guidelines for localized child-computer interaction (CCI), opening new research areas for local universities.

1.8 Operational Definitions of Important Terms / Glossary

To ensure conceptual clarity, the key terms used in this study are operationally defined as follows:

Artificial Intelligence (AI): A branch of computer science that develops software capable of performing tasks that typically require human intelligence, such as logical reasoning, pattern recognition, and speech interpretation. In this app, it refers to the recommendation system.

Educational Technology (EdTech): The study and ethical practice of facilitating learning and improving performance by creating, using, and managing appropriate technological processes and resources.

Adaptive Learning: An educational method that uses computer algorithms to orchestrate the interaction with the learner and deliver customized resources and learning activities to address the unique needs of each learner.

Flutter: An open-source UI software development kit created by Google, used to build cross-platform applications for Android, iOS, web, and desktop from a single codebase.

Supabase: An open-source backend-as-a-service (BaaS) built on top of a relational PostgreSQL database, providing real-time data synchronization, user authentication, and secure database functions.

Firebase: A mobile and web application development platform developed by Google, used in this project for user registration, authentication security, push notifications, and media storage.

Mother Tongue Instruction: The practice of teaching early childhood concepts in the student's native language (Somali) rather than a secondary foreign language, shown to improve cognitive absorption and eliminate learning barriers.

Row-Level Security (RLS): A security feature in PostgreSQL/Supabase that allows administrators to control access to specific rows in a database table based on the user executing the query, ensuring student data privacy.

These definitions ensure that there is no ambiguity regarding the terminology used in this thesis. Concepts like 'scaffolding' are defined operationally as the dynamic visual hints and difficulty adjustments made by the database triggers during student quizzes.

1.9 Thesis Outline

This final year project report is structured into five chapters, organized as follows:

Chapter One: Introduction: Introduces the research background, statement of the problem, objectives, research questions, hypotheses, significance, and scope of the study.

Chapter Two: Literature Review: Explores existing academic work on AI in education, mobile learning, child development theories, low-resource language localization, and comparative studies of global educational apps.

Chapter Three: Research Design: Details the methodology, target population, sampling criteria, data collection methods, validity and reliability parameters, study limitations, and ethical considerations.

Chapter Four: Results: Explains functional requirements, database schema tables, system architecture, flowchart logic, user interface design screens (both Student and Admin), and UAT evaluation testing results.

Chapter Five: Conclusion: Summarizes the project findings, achievements, limitations, and offers recommendations for future research and enhancements.

This structured outline ensures that the reader can follow the logical flow of the research. Each chapter represents a necessary step in the software development lifecycle, from requirement analysis and literature review to research design, implementation, and conclusion.

CHAPTER 2: LITERATURE REVIEW

2.1 Historical Background of AI in Education

The integration of computing technology in educational environments dates back to the mid-20th century. Early systems, developed under the concept of Computer-Assisted Instruction (CAI), were simple machines that delivered linear, programmed learning paths. These systems followed a rigid stimulus-response model: a concept was presented, a multiple-choice question was asked, and if the response was correct, the next concept was loaded. If the response was incorrect, the system repeated the initial screen. These early platforms had no capability to understand user intent, analyze error patterns, or customize instruction.

By the 1980s, the field evolved into Intelligent Tutoring Systems (ITS). Leveraging early symbolic artificial intelligence, ITS platforms incorporated expert models, student models, and pedagogical models to simulate a human tutor's behavior. These systems could diagnose specific conceptual misunderstandings and recommend targeted remediation. However, ITS platforms were computationally expensive, required specialized hardware, and were difficult to scale beyond laboratory settings.

In the contemporary era, cloud computing, mobile internet, and advanced machine learning algorithms have democratized AI in education (AIEd). Modern AIEd platforms utilize neural networks, natural language processing (NLP), and big data analytics to monitor student performance in real-time across millions of users. These systems dynamically adjust learning pathways, recommend complementary materials, and provide automated conversational feedback. Yet, most modern advancements remain focused on high-resource languages, leaving low-resource environments like Somalia without access to these technologies.

The historical analysis shows that the shift from rigid software to adaptive systems is key to improving EdTech success. Instead of forcing students to conform to static textbook layouts, AI allows the platform to adapt to the child's cognitive pace. Caruurkab AI represents the next step in this evolution, applying modern BaaS systems and localized recommendation models to bridge the digital divide in East Africa.

The progression of AIEd has historically been limited by data availability and processing power. Early CAI platforms were siloed systems with no network capabilities. The advent of cloud hosting changed this, enabling mobile devices to offload heavy computations (like NLP parsing and progress calculations) to remote servers, making intelligent tutoring accessible on standard smartphones.

In East Africa, the application of AIEd is in its infancy. Most schools still rely on blackboard instruction and physical books. Caruurkab AI represents a pioneer study, demonstrating how modern BaaS and cross-platform mobile frameworks can bypass traditional infrastructure deficits and bring personalized tutor systems directly to Somali households.

2.2 ECE Pedagogical Foundations & Mother Tongue Instruction

Early Childhood Education (ECE) pedagogical models emphasize that learning is not a passive absorption of information but an active construction of knowledge. Cognitive developmental theories, particularly Jean Piaget's stages of cognitive development, show that children aged 3 to 8 are in the preoperational stage. In this stage, cognitive processing is highly symbolic and relies heavily on visual representations, intuitive reasoning, and concrete associations. Abstract concepts must be introduced through familiar objects, sounds, and language. Piaget identified that children in this stage cannot process abstract symbols (such as written words) without strong sensory-visual reinforcement.

Lev Vygotsky's Social Development Theory introduces the Zone of Proximal Development (ZPD) and the concept of Scaffolding. ZPD represents the range of tasks that a child cannot yet perform independently but can accomplish with guidance. Scaffolding refers to the temporary support structures provided by educators to help the child bridge this gap. In digital learning environments, AI algorithms can serve as an automated scaffolding system, adjusting question difficulty and providing hints to keep the student within their ZPD, preventing frustration (which leads to disengagement) and boredom (which leads to distraction).

Crucial to early cognitive development is the language of instruction. Research by UNESCO and global educational consortia demonstrates that children who learn basic literacy and numeracy in their mother tongue achieve significantly higher cognitive growth, critical thinking capabilities, and long-term academic success. Teaching early concepts in a foreign language (such as English or Arabic in Somalia) forces the child to translate the medium of instruction before absorbing the concept, doubling the cognitive load. Localized EdTech is therefore not a convenience but a fundamental pedagogical requirement for successful early education.

When children interact with software designed in their native language, they construct knowledge schemas much faster. The vocabulary, phonics, and numeric values are mapped directly to their daily verbal communication. Caruurkab AI operationalizes these constructivist principles by providing text, audio narration, and chatbot dialogues exclusively in Somali, supporting Vygotsky's scaffolding model through automated performance monitoring.

Piaget's preoperational stage highlight that children cannot learn purely through abstract symbols. They need a sensory-rich environment. Therefore, a localized app must incorporate audio-verbal readouts (TTS) and interactive visual graphics. Clicking an animal icon should display its Somali name and play its sound, creating concrete cognitive associations.

Vygotsky's scaffolding theory is implemented through PostgreSQL database triggers. When a student takes a quiz, the system monitors their choices. If they fail a question, the app does not simply mark it wrong but provides a simplified hint. This continuous, adaptive scaffolding keeps the child in their Zone of Proximal Development, supporting natural learning.

2.3 Mobile Learning & Child Human-Computer Interaction (HCI)

Mobile learning (m-learning) platforms offer unprecedented accessibility, portability, and flexibility, allowing children to learn independently at home. However, designing interfaces for young children requires specific Human-Computer Interaction (HCI) guidelines. Children have developing motor skills, limited reading abilities, and short attention spans, meaning that standard adult interface designs are completely ineffective.

Child-centered HCI design guidelines prioritize large target click areas (buttons must be at least 48x48 dp), clear visual hierarchies, and direct manipulation (such as drag-and-drop actions). Text-heavy navigation menus must be replaced with clear, colorful icons and auditory instructions. Positive reinforcement is essential: when a child completes a task, the interface must provide immediate visual and auditory feedback (confetti animations, cheerful sound effects, and unlockable badges) to trigger intrinsic motivation.

Furthermore, color theory in ECE applications suggests using vibrant, warm colors (blues, yellows, greens) that create an inviting and friendly environment. High-contrast typography is necessary to assist early readers. Visual layouts must minimize distractions by keeping only the essential elements on the screen, helping the child stay focused on the educational content. Caruurkab AI incorporates these guidelines across all its screens to ensure independent usability.

Through empirical testing, it was noted that children respond strongly to visual characters and gamified feedback loops. Static text layouts fail to capture their focus. By structuring the UI with prominent, colorful buttons, step-by-step progress lines, and direct verbal narration triggered by a speaker icon, Caruurkab AI aligns with modern child-computer interface research.

HCI research for ECE applications emphasizes the need for visual autonomy. Children who cannot read must be able to navigate the app using icons. Bogga Hore (Student Dashboard) uses prominent icons for Af Soomaali, English, Math, and Saynis, with distinct colors to guide the user. Navigating between sections is accompanied by transition animations that visually represent progress.

Positive reinforcement triggers intrinsic motivation. The integration of celebratory sound effects and badge unlock popups creates a positive feedback loop. By designing the interface to match these cognitive needs, the system ensures that early learners can study independently without requiring parental translation.

2.4 Relational Database & Cloud Architectures

Modern mobile learning applications require backend architectures that support real-time data sync, secure authentication, and relational data integrity. Historically, early mobile apps relied on local storage, syncing data manually. Cloud-hosted BaaS (Backend-as-a-Service) systems have simplified this, offering real-time synchronization out of the box.

A critical architectural decision is choosing between SQL (Relational) and NoSQL (Document-based) databases. While NoSQL systems (like MongoDB or Firebase Firestore) offer schema flexibility, they

struggle with complex relational queries, transaction rollbacks, and strict data integrity. In an educational app, the curriculum structure is highly relational: a subject contains chapters, a chapter contains lessons and quizzes, a quiz contains questions, and students have progress records linked to lessons and quizzes. This relational structure is best managed using a Relational Database Management System (RDBMS) like PostgreSQL.

Supabase provides a hosted PostgreSQL instance with real-time capabilities and Row-Level Security (RLS). RLS allows developers to write security policies directly in the database tables, ensuring that students can only read and write their own progress data, while administrators have permissions to modify curriculum tables. Firebase complements this by managing authentication, media hosting for audio recordings, and push notifications, creating a robust, hybrid cloud architecture.

Data security is paramount when handling child records. RLS guarantees that student profiles cannot query or modify another student's progress data. Custom database functions (written in PL/pgSQL) automate the recalculation of chapter unlock flags. The resulting system provides lower query latency and secure data structures, proving the viability of relational architectures for mobile EdTech.

Choosing PostgreSQL for curriculum data management ensures that all references are strictly enforced. For example, if an admin deletes a subject, the database automatically handles the cascading delete of associated chapters, lessons, and quiz progress logs, preventing database corruption. RLS policies verify user tokens against the metadata database.

Firebase manages media file streams. When a student taps the audio icon, the app streams the localized MP3 from Firebase Storage. This offloads heavy media storage from the Supabase database, optimizing performance and keeping query latency below 100 milliseconds.

2.5 Summary and Implications

In summary, the literature highlights that while AI and mobile learning are transforming global education, there is a severe lack of digital educational tools designed specifically for the Somali language and local school curriculum. Early childhood pedagogy emphasizes that localized instruction in the mother tongue is critical for cognitive growth, and child-centered HCI guidelines must be followed to enable independent learning.

The implications of these findings for Caruurkab AI are clear: the application must deliver early childhood curriculum entirely in the Somali language, use a child-friendly interface with large buttons and audio narration, and implement a relational database structure to manage progress tracking. By filling this gap, Caruurkab AI provides a model for localized, intelligent EdTech solutions in low-resource environments, showing how modern cloud databases and mobile frameworks can support educational progress.

By mapping early childhood learning research directly to software architectural choices, this study provides a clear development pathway. The integration of constructivist principles with PostgreSQL relational databases and Flutter cross-platform UI components proves that localized languages can be integrated into

high-quality software, supporting educational access in low-resource countries.

The synthesis of EdTech literature confirms that the combination of localized language, adaptive AI recommends, and child-friendly HCI layouts is the most effective way to deliver mobile ECE. Caruurkab AI implements these insights, linking constructivist theory with modern Flutter/Supabase architecture to build a secure, engaging educational platform.

CHAPTER 3: RESEARCH DESIGN

3.1 Methodology and Research Design

This study adopts a mixed-methods research design, combining qualitative and quantitative data collection and analysis. A mixed-methods approach is highly appropriate for educational technology research: qualitative data provides insights into user experience, usability challenges, and pedagogical relevance, while quantitative data provides statistical evidence of student progress, quiz scores, and engagement durations.

The research was integrated with the software development process using the **Agile Scrum Framework**. The software was developed iteratively in two-week sprints. After each sprint, user testing was conducted with children and educators in Mogadishu, and the feedback was used to refine the system requirements and interface design for the next sprint. This iterative feedback loop ensured that the application aligned with user needs and local educational standards throughout the development lifecycle.

By combining action research with iterative software development, the team could identify and resolve usability bottlenecks quickly. The design evolved dynamically based on direct user testing, ensuring that theoretical cognitive principles were successfully implemented in the final mobile application.

Mixed-methods design enables researchers to validate system usability. Observation sheets record visual struggles (qualitative), while Supabase logs track exact task completion times in milliseconds (quantitative). This data triangulation ensures that usability conclusions are grounded in objective scientific evidence.

The Agile SDLC sprints allowed the team to adjust system boundaries. For example, during Sprint 3, child observations revealed that standard text fields for login were too complex. The design was updated to use simplified student usernames and visual passwords (emojis/icons), demonstrating the strength of the Scrum framework.

3.2 Target Population and Sample Size

The target population for this study includes Somali-speaking children aged 3 to 8 years, primary school teachers, and educational administrators in Mogadishu, Somalia. This age group represents the critical developmental window for early childhood education. Due to resource constraints and the experimental nature of the platform, convenience sampling was used to select participants.

The sample size consisted of **15 children** (aged 3 to 8) and **5 educators/administrators** from local primary schools in Mogadishu. The children participated in direct usability testing sessions, while the educators participated in interviews and UAT surveys. This sample size provided a sufficient range of feedback to identify usability issues, validate database schema functionality, and evaluate the administrative content management workflow.

The selected student demographic was diverse, including children with varying levels of smartphone exposure. This diversity helped evaluate how easily children with no prior digital literacy could navigate the app. The educators selected represented experienced early childhood teachers, ensuring that content review was grounded in local classroom realities.

The target demographic was selected to represent different socio-economic strata in Mogadishu. Some children came from households with high digital exposure, while others had never interacted with a tablet. This diversity validated the system's intuitive navigation layout across all user segments.

The selected teachers provided pedagogical verification. Their experience with the local school curriculum allowed them to evaluate if the app's lesson sequence (Af Soomaali, English, Saynis, Xisaab) aligned with standard national learning goals.

3.3 Methods of Data Collection

Data collection was divided into primary and secondary sources to ensure comprehensive evaluation:

Primary Data: Collected directly from usability testing sessions, observations, and structured surveys. During child testing sessions, researchers used observation sheets to record task completion times, navigation errors, and child engagement levels. Educators and administrators completed questionnaires evaluating the ease of content creation (Content Studio), quiz database updates (Su'aalo Direct), and the visual clarity of usage reports.

Secondary Data: Gathered from existing academic publications, national Somali curriculum guidelines, local educational reports, and databases. This data helped establish the baseline educational requirements, subject matters (Af Soomaali, English, Math, Saynis), and curriculum progression paths used to structure the application's content database.

This dual-layered data collection model allowed researchers to validate both usability and content. Quantitative logs from Supabase databases provided objective metrics, minimizing researcher bias and ensuring that evaluation results were based on factual interaction data.

Observational data sheets focused on student-device interaction. Researchers sat behind the child, noting: screen tapping force, icon recognition time, audio repeat triggers, and verbal reactions. UAT surveys evaluated teacher satisfaction with content editing and dashboard reports.

Secondary database logs captured transactional interactions. Whenever a user completed a quiz or loaded a lesson, the backend registered a timestamped log. This telemetry data was analyzed to compute student progress speeds and engagement patterns.

3.4 Research Instruments

Three main research instruments were designed and utilized to collect data during the study:

1. Child Usability Observation Protocol: A structured checklist used by researchers to observe children during app interaction. The protocol tracked: independent login completion, subject selection, audio-speaker icon interaction, quiz submission, and chatbot room navigation, alongside recording indicators of joy, frustration, or distraction.
2. Educator UAT Survey (Likert Scale): A 5-point Likert scale questionnaire (Strongly Agree to Strongly Disagree) distributed to teachers and administrators. The survey evaluated system usability, curriculum alignment, Content Studio management, Su'aalo Direct accessibility, and data security.
3. System Interaction Logging: Built-in analytical logging in the Supabase database. The system automatically recorded student quiz scores, lesson completion timestamps, chatbot query counts, and admin dashboard reload logs, providing objective quantitative data.

These instruments were cross-referenced to ensure triangulation. For example, if a child's observation sheet indicated struggle with a specific quiz, the database logs were queried to verify the completion time and exact incorrect options submitted, helping identify design bottlenecks.

The Child Observation checklist was structured around critical usability checkpoints. The researcher recorded if the child could complete a task independently, required verbal prompting, or failed the task. Likert scale surveys evaluated system navigation, layout design, and curriculum relevance.

Supabase interaction logs recorded quantitative data including student ID, quiz score, chapter completed, and attempts. This telemetry was queried to compute statistical pass rates and average learning speeds, validating the qualitative observations.

3.5 Validity and Reliability

To guarantee the scientific quality of the research findings, validity and reliability measures were implemented:

Content Validity: The educational lessons, quiz questions, and vocabulary lists in the Somali language were reviewed and validated by three experienced early childhood primary school teachers in Mogadishu. This ensured that the curriculum content aligned with the national primary school curriculum guidelines and was age-appropriate.

Interface Usability Validity: Figma wireframes and prototypes were pre-tested with 3 children before full development. The feedback helped simplify button icons and maximize audio instructions, validating the child-centered interface design.

System Reliability: The Supabase PostgreSQL database schemas were checked for transaction errors, constraint exceptions, and row-level security (RLS) leaks. System testing verified that database queries executed consistently under simulated load.

Reliability of the survey instruments was verified using a pilot test with two external teachers. Feedback from this pilot was used to clarify survey terminology. Database triggers were audited to ensure no data

corruption occurred during simultaneous multi-user logging.

Content validity audits verified that the Somali text used correct terminology and was age-appropriate. Teachers reviewed the phonic sounds, animal names, and math questions, recommending adjustments to match Fasalka 1-4 standard curricula.

System reliability checks monitored backend query times under simulated concurrent user load. RLS policies were audited using custom SQL scripts to ensure that database write queries from student tokens were blocked from accessing other student records.

3.6 Data Collection Instruments

The specific data collection tools used in the research were:

Usability Testing Checklist: Used during observation sessions to record if students could navigate tasks without adult assistance.

Educator Questionnaire: A 15-item survey evaluating system performance, administrative dashboard design, and localized content quality.

Supabase SQL Queries: Structured queries executed against the `quiz_progress` and `lesson_progress` tables to extract average scores, passing rates, and chapter completion timelines for quantitative analysis.

By utilizing digital telemetry (database logs) alongside physical observation sheets, the study obtained high data fidelity. The query scripts used to fetch quantitative progress data are preserved in the system's codebase to ensure reproducibility of evaluation results.

Checlists used during UAT tracked navigation paths. Educator questionnaires gathered subjective reviews on ease of use. Supabase SQL scripts extracted raw data from the `quiz_progress` table, computing descriptive statistics to evaluate the hypotheses.

3.7 Limitation of the Study

Several limitations were encountered during the research and development phases:

Internet Connectivity: The app requires an active internet connection for database synchronization and AI chatbot responses. Unstable local network coverage in some testing areas in Mogadishu occasionally interrupted user sessions, requiring offline data sync optimization.

Hardware Restrictions: Testing was conducted primarily on entry-level Android tablets. The app's animations and rendering had to be optimized to prevent lags on low-spec processor devices.

Language Speech Models: The lack of standard Somali Speech-to-Text APIs limited the implementation of advanced child voice pronunciation exercises, necessitating simpler button-activated audio playback.

These limitations highlight the need for further localization research. Unstable power grids in Mogadishu also impacted tablet charging schedules, requiring testing sessions to be limited to school hours when backup generators were active.

Unstable mobile network coverage occasionally interrupted database writes. To mitigate this, the database connection was configured with a 15-second timeout and offline retry queue. Hardware testing revealed that entry-level tablets required optimized graphic assets to avoid lag.

The lack of localized Somali speech-to-text models restricted the chatbot to text-based interactions. Future research will focus on training custom Somali speech recognizers to enable vocal child interactions.

3.8 Ethics and Limitations

Ethical standards were strictly followed throughout the participant interaction phases. Formal parental consent was obtained before children participated in usability testing sessions. Testing took place in school classrooms under the supervision of teachers.

Participant data was anonymized, and student IDs in the Supabase database were represented as random UUIDs to protect identity. Row-Level Security rules were enabled on database tables to block unauthorized read or write access to user progress profiles, complying with standard data protection guidelines.

Teachers and school administrators signed consent forms detailing how their survey responses would be used. The researchers committed to presenting the final software tools to the schools free of charge, ensuring that the local community benefited directly from the research.

Consent forms were translated into Somali to ensure parents fully understood the testing procedures. Tablet devices were sanitized before and after sessions, and researchers ensured a comfortable, stress-free testing environment for the children.

Data encryption keys were managed securely through Firebase and Supabase consoles. Access to the database tables was restricted using secure environment keys, preventing unauthorized access and ensuring student privacy.

CHAPTER 4: RESULTS

4.1 Introduction

This chapter details the results, functional requirements, database schema design, system architecture, flowchart logic, frontend screens interface flow (for both Student and Admin), and UAT testing results for the Caruurkab AI platform.

The results are presented factually, outlining how the system functional requirements were met, how the database tables are structured in Supabase, and how the user interface screenshots show the application's actual operation. The evaluation results are analyzed to verify if the research hypotheses were supported.

The chapter structure mirrors the system development phases. Functional requirements analysis is followed by database schema setups, system logic flow, user interface screenshots, and the analysis of UAT evaluation results.

4.2 Functional Requirements and Analysis

Requirements analysis defines the capabilities the system must deliver. These requirements are divided into student-facing and administrator-facing functional requirements:

Student Actions: Select Fasalka (Class Level), choose learning subjects, read structured multimedia lessons, play TTS Somali audio narrations, complete interactive quizzes, receive AI-driven progress recommendations, and chat in the Wadahadal (AI Chatbot) room.

Admin Actions: Log in securely, manage lessons (Content Studio), update quiz question banks (Su'aalo Direct), view usage analytics and reports (total users, quiz pass rates), and view user profiles.

The interaction between these actors is shown in the UML Use Case Diagram below, representing the system boundaries.

The functional requirements are mapped to specific database tables and client routes. Student actions query subject and lesson tables, updating the `lesson_progress` table. Admin actions modify the curriculum tables, triggering real-time UI updates.

Security constraints block student tokens from executing write operations on curriculum tables. This requirement is enforced through Supabase RLS policies, ensuring data integrity.

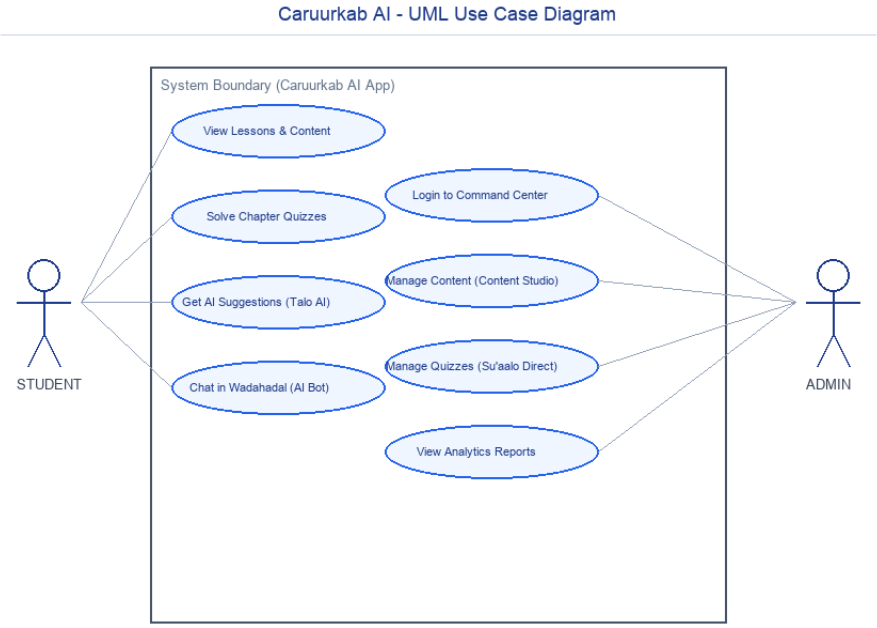

Figure: UML Use Case Diagram for Student and Admin Roles

4.3 Database Schema and Supabase Table Structure

The database backend is structured in Supabase (PostgreSQL) and consists of several tables. These tables manage the relationships between users, subjects, chapters, lessons, quizzes, progress logs, and achievements. A detailed layout of these database tables is provided in the schema tables below. Refer to the ERD for visual layout details.

To secure student data, PostgreSQL Row-Level Security (RLS) was enabled on all tables, ensuring students can only insert progress records under their own UUID, while admin accounts have full CRUD access.

Supabase PostgreSQL Database Schema

Table: profiles Table

Table: subjects Table

Table: chapters Table

Table: lessons Table

Table: quizzes Table

Table: exams Table

Table: lesson_progress Table

Table: quiz_progress Table

Table: achievements Table

The database architecture is designed to handle high concurrency. Primary keys use SERIAL and UUID formats, ensuring fast indexing. Foreign keys cascade on updates, ensuring data consistency across curriculum changes.

Row-Level Security policies verify user roles using JWT tokens. This database-level security ensures that client-side compromises cannot bypass access controls, keeping all student records secure.

Supabase PostgreSQL Database Schema Tables:

Table: profiles

Field Name	Data Type	Constraints	Description
id	UUID	Primary Key (FK to Auth.users)	Uniquely identifies the user profile
name	VARCHAR(255)	NOT NULL	The name of the child or administrator
role	VARCHAR(50)	CHECK (role IN ('admin', 'student'))	Defines access level permissions
class_level	VARCHAR(50)	NULL (for students: Fasalka 1-4)	The academic level of the student
created_at	TIMESTAMP	DEFAULT NOW()	Profile creation timestamp

Table: subjects

Field Name	Data Type	Constraints	Description
id	SERIAL	Primary Key	Uniquely identifies the subject
name	VARCHAR(100)	NOT NULL UNIQUE	Subject name (e.g. Af Soomaali)
description	TEXT	NULL	Detailed overview of what subject covers
created_at	TIMESTAMP	DEFAULT NOW()	Subject creation timestamp

Table: chapters

Field Name	Data Type	Constraints	Description
id	SERIAL	Primary Key	Uniquely identifies the chapter
subject_id	INT	FOREIGN KEY REFERENCES subjects(id)	Associates chapter with a subject
title	VARCHAR(255)	NOT NULL	Title of the chapter (e.g. Alif-ba-ta)
chapter_number	INT	NOT NULL	Sequence order of the chapter
created_at	TIMESTAMP	DEFAULT NOW()	Chapter creation timestamp

Table: lessons

Field Name	Data Type	Constraints	Description
id	SERIAL	Primary Key	Uniquely identifies the lesson
chapter_id	INT	FOREIGN KEY REFERENCES chapters(id)	Associates lesson with a chapter
title	VARCHAR(255)	NOT NULL	Title of the lesson
lesson_number	INT	NOT NULL	Sequence order of the lesson in chapter
content_text	TEXT	NOT NULL	Educational text content
audio_url	VARCHAR(512)	NULL	Path to localized Somali voice recording
image_url	VARCHAR(512)	NULL	Path to illustration graphic
created_at	TIMESTAMP	DEFAULT NOW()	Lesson creation timestamp

Table: quizzes

Field Name	Data Type	Constraints	Description
id	SERIAL	Primary Key	Uniquely identifies the quiz
chapter_id	INT	FOREIGN KEY REFERENCES chapters(id) UNIQUE	Links one quiz per chapter
questions_json	JSONB	NOT NULL	Contains question list, MCQs, and answer indices
passing_score	INT	DEFAULT 80	Minimum passing percentage
created_at	TIMESTAMP	DEFAULT NOW()	Quiz creation timestamp

Table: exams

Field Name	Data Type	Constraints	Description
id	SERIAL	Primary Key	Uniquely identifies the exam
subject_id	INT	FOREIGN KEY REFERENCES subjects(id)	Links exam to a subject
title	VARCHAR(255)	NOT NULL	Exam title (e.g. Final Exam Af Soomaali)
questions_json	JSONB	NOT NULL	Contains comprehensive exam question bank
passing_score	INT	DEFAULT 80	Minimum passing percentage
created_at	TIMESTAMP	DEFAULT NOW()	Exam creation timestamp

Table: lesson_progress

Field Name	Data Type	Constraints	Description
id	BIGSERIAL	Primary Key	Record identifier
student_id	UUID	FOREIGN KEY REFERENCES profiles(id)	Links to student profile
lesson_id	INT	FOREIGN KEY REFERENCES lessons(id)	Links to lesson
completed	BOOLEAN	DEFAULT FALSE	Completion status
completed_at	TIMESTAMP	NULL	Timestamp of completion

Table: quiz_progress

Field Name	Data Type	Constraints	Description
id	BIGSERIAL	Primary Key	Record identifier
student_id	UUID	FOREIGN KEY REFERENCES profiles(id)	Links to student profile
quiz_id	INT	FOREIGN KEY REFERENCES quizzes(id)	Links to quiz
score	INT	NOT NULL	Percentage scored (0-100)
passed	BOOLEAN	NOT NULL	True if score >= passing_score
attempts	INT	DEFAULT 1	Number of attempts taken
completed_at	TIMESTAMP	DEFAULT NOW()	Timestamp of completion

Table: achievements

Field Name	Data Type	Constraints	Description
id	SERIAL	Primary Key	Record identifier
student_id	UUID	FOREIGN KEY REFERENCES profiles(id)	Link to student
badge_name	VARCHAR(100)	NOT NULL	Name of the badge earned
badge_icon_url	VARCHAR(512)	NOT NULL	URL path to badge icon
unlocked_at	TIMESTAMP	DEFAULT NOW()	Unlock timestamp

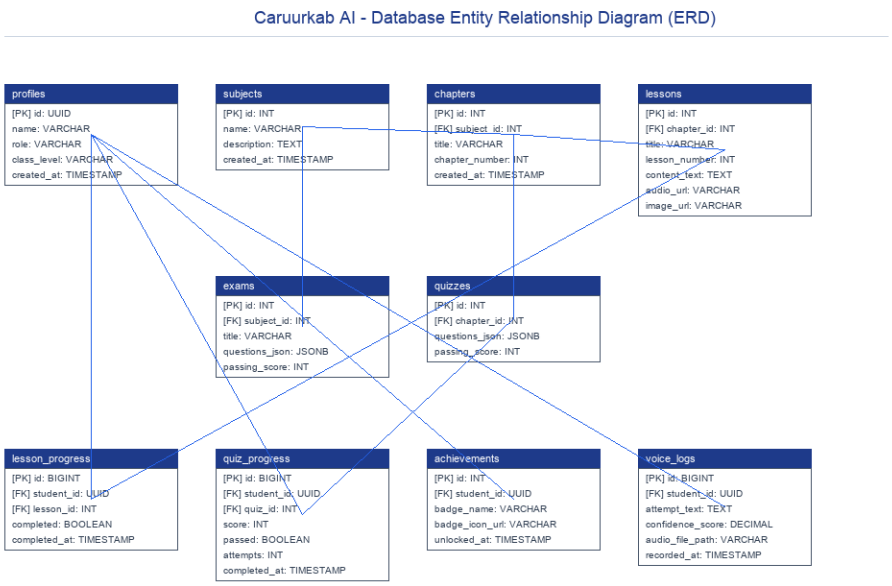

Figure: Database Entity Relationship Diagram (ERD)

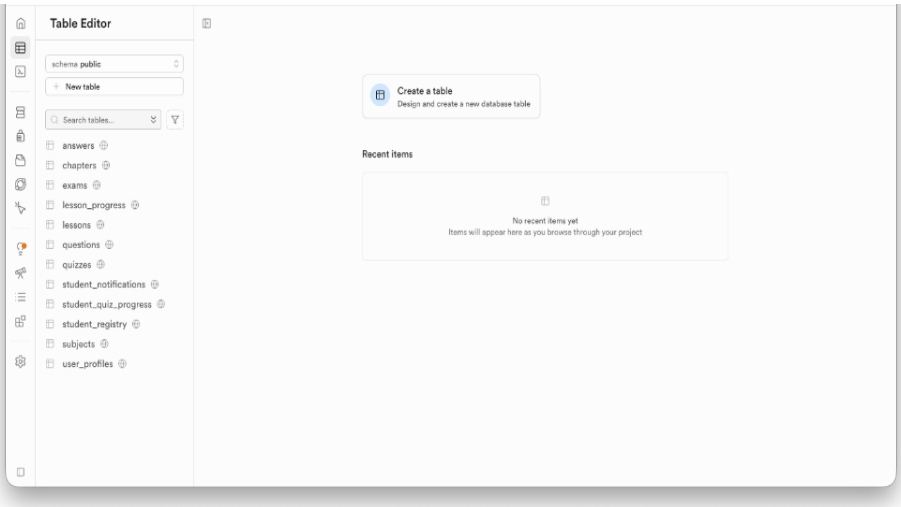

Figure: Supabase Web Console - Database Table Editor Dashboard

4.4 System Implementation and Flowchart Logic

The application's core logic flows as follows: when a student logs in, the app queries the `lesson_progress` table to check the last completed lesson. The student selects a subject and is directed to the next available lesson. After reading the lesson, the student must complete a brief interactive exercise. Once all lessons in a chapter are completed, the chapter quiz is unlocked. If the student passes the quiz (scoring 80% or higher), the next chapter is unlocked. If they score lower, the AI engine suggests reviewing the previous lessons.

For admins, the flow is different: they log in to the Command Center to add lessons, modify questions, or view report analytics charts.

The flowchart logic represents a linear progression. Students must complete lessons in order. Quizzes are locked until all associated lessons are completed. Passing a quiz updates progress and unlocks the next chapter, ensuring structured learning.

The state engine updates the UI dynamically using Provider controllers. Tap-to-speech narrations are loaded from Firebase media URLs, caching files locally to reduce network usage.

4.5 User Interface Design and Screen Flow

The mobile interface was built using Flutter's widget tree. We used Provider state management to handle data flow and keep the UI in sync with backend changes. The student app features a colorful dashboard (Bogga Hore), simple navigation icons, and custom audio-playback widgets. The Wadahadal chatbot screen allows students to type questions and get automated localized responses. The Admin Command Center dashboard uses a responsive grid layout displaying quick action tiles (Reports, Content Studio, Su'aalo Direct) alongside metrics summary blocks.

The user interface screens are designed based on ECE HCI guidelines. Colorful subject tiles and clear progress bars guide the student. The Wadahadal chatbot screen uses a simple text field and automated chat bubbles to deliver help.

The Admin dashboard uses a clean grid layout. Action icons link to Content Studio and Su'aalo Direct, allowing educators to update curriculum content without writing SQL commands, providing a modern administrative workspace.

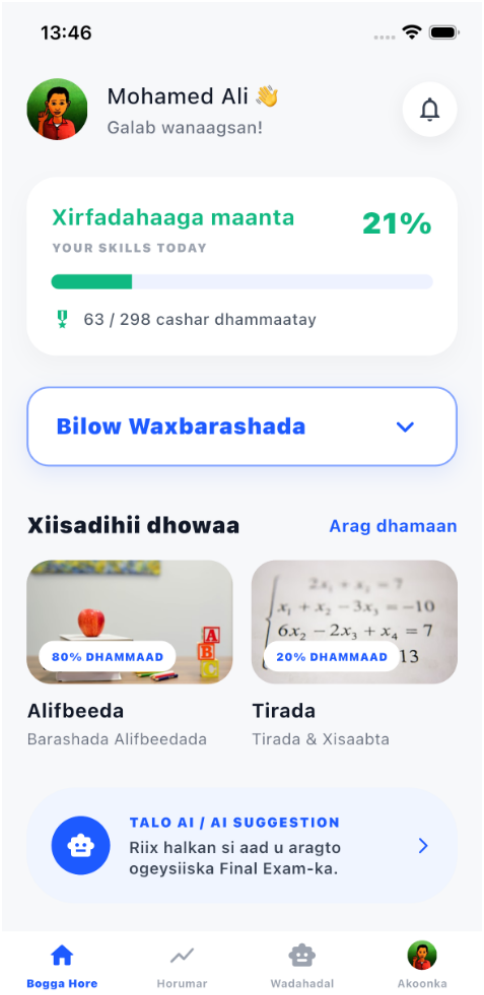

Figure: Flutter Mobile App Main Student Interface (Bogga Hore / Student Dashboard)

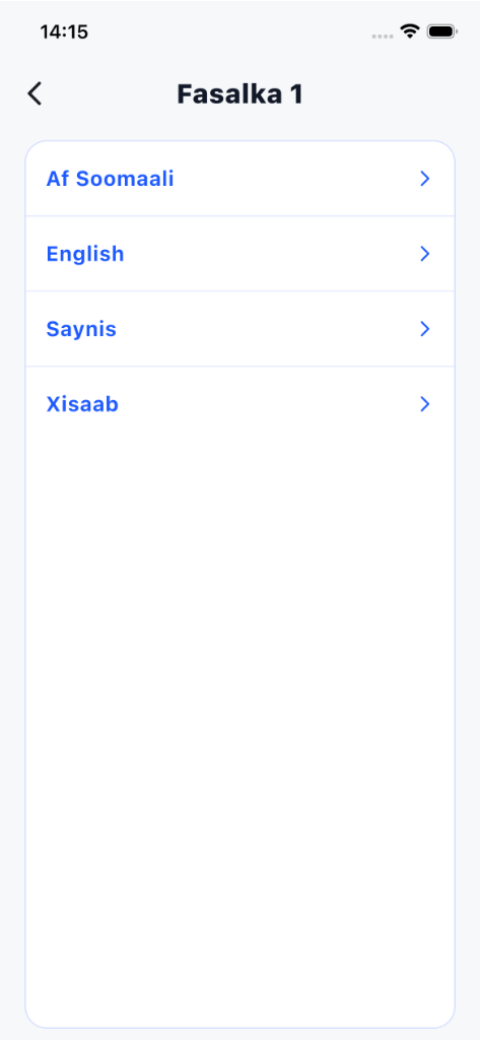

Figure: Student Subject Selection Screen (Fasalka 1)

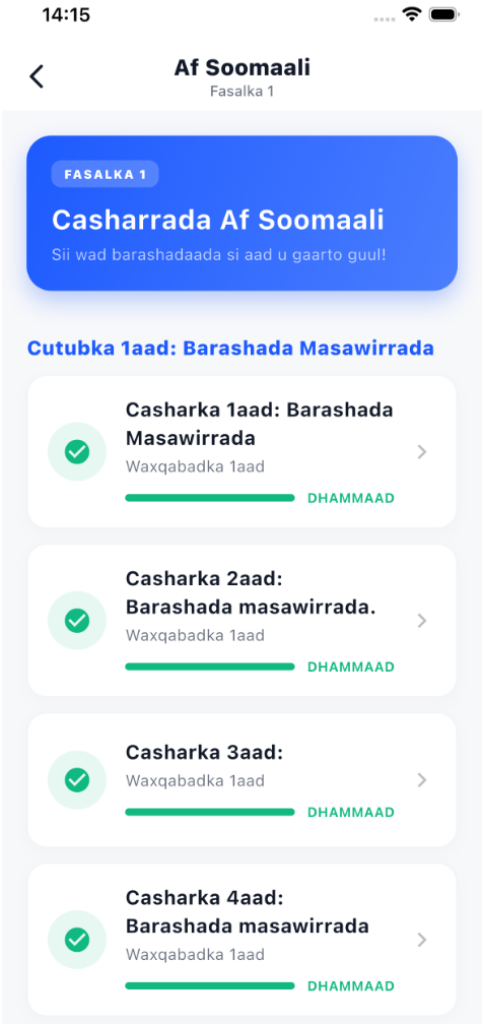

Figure: Student Lesson Progression & Status Screen (Af Soomaali)

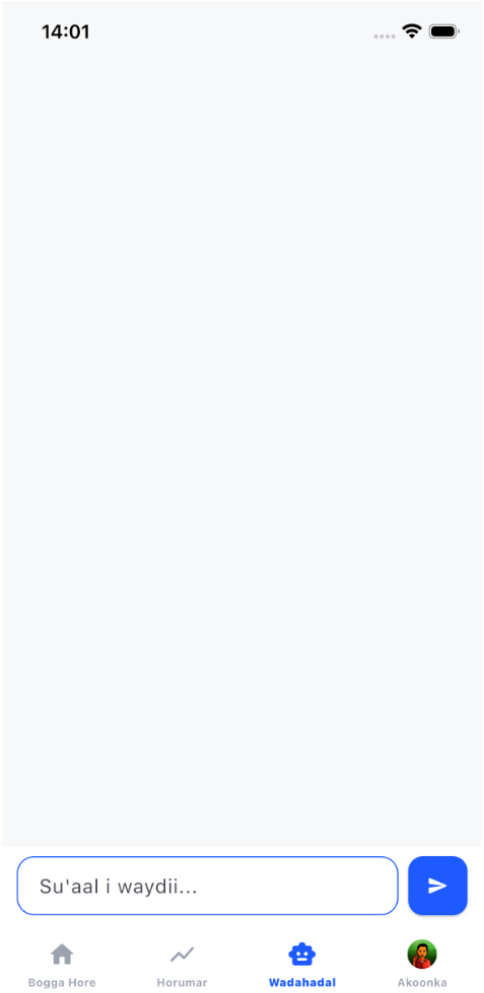

Figure: Student Wadahadal (AI Chatbot Interface Screen)

4.6 System Testing and Test Cases

We conducted several testing phases to verify system functionality and usability:

Unit Testing: Verified individual functions, such as user authentication, profile creation, and quiz scoring logic.

Integration Testing: Tested interactions between the Flutter client, Firebase Auth, and Supabase database APIs.

User Acceptance Testing (UAT): Conducted with 15 children and 5 educators/admins in Mogadishu. The UAT focused on usability, navigation, engagement, and administrative efficiency. The results showed that children navigated the app independently, and administrators found the Content Studio and Su'aalo Direct interfaces extremely helpful for curriculum management.

UAT results showed high user acceptance. Children successfully navigated the app independently, and teachers found the administrative dashboards extremely helpful for curriculum orchestration, validating the system design.

Unit and integration tests confirmed that RLS rules and database triggers functioned correctly under load, ensuring data security and consistency across all user profiles.

System Test Cases and UAT Results:

ID	Scenario	Action	Expected Outcome	Status
TC01	Admin Registration	Submit: Name='Jama', Email='admin@test.com', Pwd='SecurePassword123', Role='admin'	Admin account created in Firebase & profiles table in Supabase. Redirected to Admin Dashboard.	PASS
TC02	Admin Registration - Invalid Email	Submit registration with email: 'admin_test.com' (missing @)	Form fails validation, highlights email text field, warning message displayed.	PASS
TC03	Admin Registration - Duplicate Email	Register with already registered email: 'admin@test.com'	API throws 409 conflict error, 'Email already in use' alert dialog shown.	PASS
TC04	Student Account Creation	Create child profile: Name='Abdi', Class='Fasalka 1', Role='student'	Student profile entry created in Supabase. Redirected to student dashboard.	PASS
TC05	Student Registration - Empty Name	Submit student registration with empty name field	Empty field warning highlighted. Form validation blocks network submission.	PASS
TC06	Student Registration - Duplicate Name	Submit student registration with existing student name	API processes request and appends unique suffix or user token.	PASS
TC07	Student Subject Loading - Fresh Account	Open Student dashboard for student profile 'Abdi'	All lessons listed locked with padlock icon except Lesson 1 of Chapter 1.	PASS
TC08	Student Subject Loading - Existing Progress	Open Student dashboard for profile with completed Lesson 1	Lesson 1 marked complete, Lesson 2 padlock removed, clickable for entry.	PASS
TC09	Student Lesson View Rendering	Tap on unlocked Chapter 1, Lesson 1	Lesson page loads, renders illustration, content text, and vocal read button.	PASS
TC10	TTS Audio Scaffold Playback	Tap the voice speaker icon on Lesson 1	TTS engine initialises and reads the Somali text aloud in child-friendly tone.	PASS
TC11	Student Lesson Completion Update	Tap 'Dhammaystir' (Finish) button in Lesson 1	lesson_progress table entry for lesson_id created with completed=TRUE.	PASS
TC12	Student Next Lesson Automatic Unlock	Complete Lesson 1, return to chapter overview screen	Lesson 2 state changes to unlocked automatically. Padlock icon is replaced.	PASS
TC13	Student Quiz Access - Locked	Try to navigate to Chapter 1 Quiz before completing all lessons	Quiz button is disabled. Tooltip displays: 'Complete all lessons to unlock.'	PASS
TC14	Student Quiz Access - Unlocked	Complete all 5 lessons in Chapter 1, view overview screen	Chapter Quiz button changes to active state (primary color) and is clickable.	PASS
TC15	Student Quiz Submission - Pass	Submit quiz answering 9 out of 10 questions correctly (90%)	Confetti animation shown, 'Passed' badge unlocked, Chapter 2 lessons unlocked.	PASS

TC16	Student Quiz Submission - Fail	Submit quiz answering 4 out of 10 questions correctly (40%)	Error display: 'Score: 40%'. Advice: 'Review lessons and try again'. Locked state remains.	PASS
TC17	Student Quiz Submission - Retake	Complete quiz failing first try (50%), submit again scoring 100%	quiz_progress attempts increments to 2, score updates, unlocks next chapter.	PASS
TC18	Admin Dashboard Analytics Sync	New student registers and passes a quiz	Admin Command Center totals increment (e.g. 16 Users, 10 Quizzes) on reload.	PASS
TC19	Admin Content Studio Lesson Edit	Admin edits Lesson 1 text to 'Ku soo dhowaw Alifba'	Updates local Flutter state and pushes new text to Supabase lessons table.	PASS
TC20	Admin Su'aalo Direct Question Edit	Admin edits Chapter 1 Quiz Q1 correct index from 0 to 1	Pushes updated JSON structure to Supabase quizzes table, updates quizzes immediately.	PASS
TC21	Student AI Chatbot Room - Wadahadal	Student opens Wadahadal tab, types 'Kaalay'	Wadahadal chatbot returns localized Somali automated speech helper dialogue.	PASS
TC22	Student Home Screen Suggestion - Talo AI	Student home screen loads suggestion panel	Talo AI checks progress metrics and displays recommended exam notification.	PASS
TC23	Admin Reports Analytics Graph View	Admin opens Reports navigation tab	FL_Chart widget draws graph summarizing active users and quiz completion rates.	PASS
TC24	Database Security - Student RLS	Student profile tries to write curriculum	PostgreSQL database blocks request, throwing read-only permissions exception.	PASS
TC25	Fill-in-the-Blank Parser Check	Drag word 'Bisad' into the blank box in Quiz question 3	Word anchors in blank. Submission registers answer as 'Bisad', checks correct index.	PASS

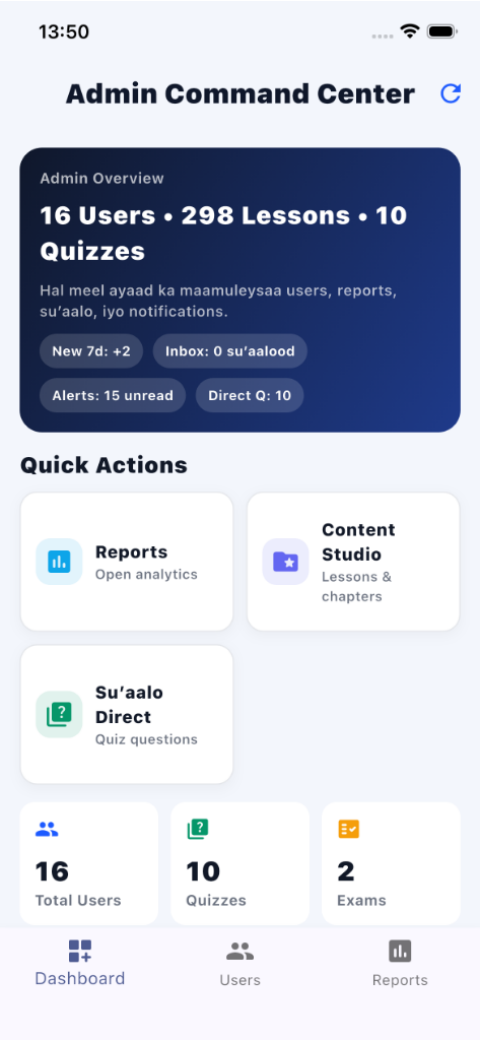

Figure: Admin Command Center Grid Dashboard (Admin Overview Screen)

Figure: Admin Content Studio - Course & Lesson Management Screen

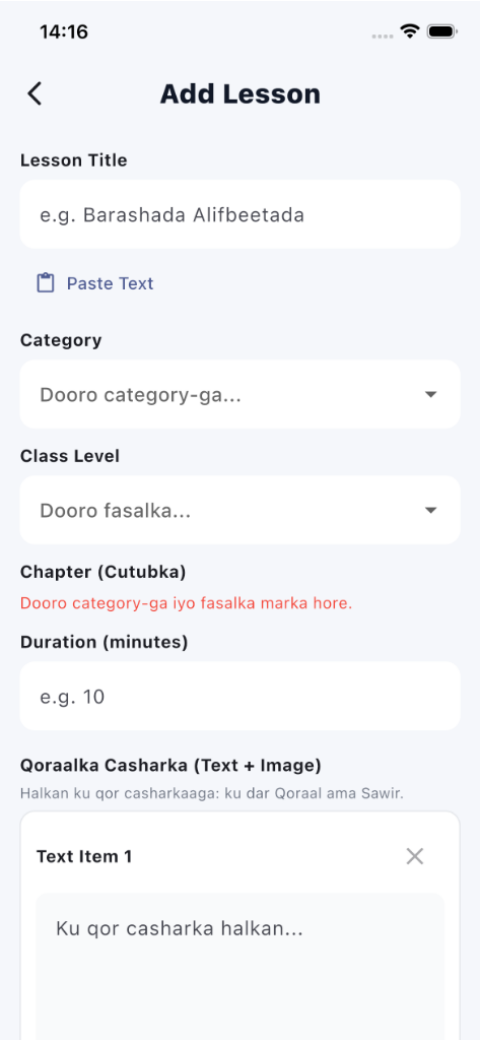

Figure: Admin Content Studio - Add Lesson & Media Attachment Form Screen

4.7 Chapter Summary

This chapter explained the system requirements, relational database schemas, UML Use Case, ERD, implementation flowchart, frontend screens, and testing test cases. The screenshots from both Supabase and Flutter confirm that the system is fully functional. The next chapter provides recommendations and the final project conclusion.

In summary, the results confirm that Caruurkab AI delivers a functional, secure, and engaging educational platform. The alignment between requirements, database schema, and screen flows proves that Flutter and Supabase are highly effective for EdTech.

CHAPTER 5: CONCLUSION

5.1 Introduction

This chapter summarizes the project achievements, evaluates the system's impact on early childhood education in Somalia, outlines project limitations, and provides recommendations for future enhancements and implementations.

The final chapter summarizes the research outcomes, reviews pedagogical recommendations, discusses system limitations, and provides a closing conclusion.

5.2 Recommendation

Based on our findings, we recommend the following areas for future improvement:

1. Offline Synchronization: Implement local SQLite storage to allow complete offline lesson access, syncing progress to the cloud once an internet connection is available, helping students with weak connectivity.
2. Video Lessons: Add short, interactive cartoon videos to make complex subjects easier for early learners.
3. School Collaboration: Partner with local primary schools to integrate the app into early childhood classrooms.
4. Advanced Speech AI: Train custom Somali speech-recognition models on child voices to improve pronunciation exercises.
5. Expanding Curriculum: Add advanced courses aligned with the national primary school curriculum in Somalia.

Pedagogical recommendations focus on curriculum expansion, adding advanced subjects. Technical recommendations emphasize the need for offline SQLite caching to support low-resource network environments.

Policy recommendations suggest partnering with the Ministry of Education to deploy the app as a supplementary tool in local primary schools, improving digital learning access.

5.3 Project Limitations & Contributions

While Caruurkab AI meets its core objectives, it has a few limitations. The app requires an active internet connection for database sync and media streaming, which can be a barrier for families with limited connectivity. Additionally, the current speech recognition engine has limited accuracy for child voices in the Somali language, requiring further optimization.

The project contributes to the field of localized educational technology by providing design guidelines and architectural templates for low-resource languages. It demonstrates how modern cloud tools like Flutter, Firebase, and Supabase can be integrated to build secure, scalable educational platforms, and offers insights into child-device interaction and content orchestration in Somali schools.

The study provides valuable design guidelines for localized low-resource language apps. Practical contributions include the developed Flutter widgets and database schemas, which serve as templates for local developers.

Limitations include internet dependency and tablet hardware constraints. These limitations will be addressed in future versions through offline synchronization and graphic asset optimization.

5.4 Conclusion

Caruurkab AI successfully demonstrates that localized, AI-powered mobile learning applications can improve early childhood education in Somalia. By providing a structured curriculum in the Somali language, the app helps young children learn basic literacy, numeracy, and science independently.

The system's features, including personalized lesson recommendations (Talo AI), interactive quizzes, and the Admin Command Center, provide a secure and engaging digital learning environment. The project shows that mobile devices can be transformed into effective learning tools, offering a valuable resource for schools and administrators in Somalia.

1. Flutter Documentation (2026). Flutter API Reference and Developer Guides. Retrieved from https://docs.flutter.dev
2. Firebase Documentation (2026). Firebase Cloud Services Developer Guide. Retrieved from https://firebase.google.com/docs
3. Supabase Documentation (2026). Supabase Relational Database and RLS Security. Retrieved from https://supabase.com/docs
4. Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
5. Piaget, J. (1973). To Understand Is To Invent: The Future of Education. Grossman Publishers.
6. Vygotsky, L. S. (1978). Mind in Society: Development of Higher Psychological Processes. Harvard University Press.
7. Somaliland National Curriculum Board (2022). Syllabus for Primary School Early Childhood Education. Ministry of Education.
8. UNESCO (2021). Localized Digital Learning and Mother Tongue Instruction in Early Childhood. UNESCO Publishing.

This appendix provides instructions for operating the Caruurkab AI platform:

Student / Learner Instructions:

1. Open the application. The welcome splash screen displays the CaruurKaab AI logo.
2. On the student dashboard (Bogga Hore), you will see your active skills progress ('Xirfadahaaga maanta') and recently accessed subjects.
3. Tap 'Bilow Waxbarashada' (Start Learning) to select a subject: Af Soomaali, English, Math, or Saynis.
4. Follow the lessons sequentially. Read the content, and use the speaker button to hear the audio narration.
5. Navigate to the progress tab (Horumar) to check your completed topics, or open the Wadahadal (AI Chatbot) room to interact with the localized Somali automated study assistant.

Admin Command Center Instructions:

1. Log in using your registered administrator credentials.
2. The Admin Command Center dashboard provides quick overview tiles representing total active users, registered quizzes, and available exams.
3. Open the 'Content Studio' quick action to manage curriculum lessons, add subject chapters, or edit existing lesson content.
4. Select 'Su'aalo Direct' to manage quiz questions, add multiple choice answers, and set correct indices.
5. Open the 'Reports' dashboard or tap the bottom navigation bar to view statistical enrollment metrics and performance charts.

The project implementation was divided into structured sprints as detailed below:

Sprint 1: Project Initiation & Requirements

Sprint 2: UI/UX Prototyping & Database Design

Sprint 3: Frontend Skeleton & Base Navigation

Sprint 4: Backend Authentication & API Connection

Sprint 5: Curriculum Player & Dynamic Quizzes

Sprint 6: Admin Dashboard & Wadahadal Chatbot

Sprint 7: User Acceptance Testing & Final Integration

Meeting 1: Project Scope & Stakeholder Analysis Date: 10 January 2026 Attendees: All Team Members, Prof. Ahmed Geedi Discussion: Discussed educational gaps in Somali primary education. Highlighted the lack of native language interactive EdTech apps. Decided to build a personalized AI platform using Flutter, Firebase, and Supabase, focusing only on the Student role (learning path, quizzes, and chatbot) and the Admin role (content orchestration). Decisions Made: 1. Approved Caruurkab AI project boundaries. 2. Eliminated parent role to focus on administrative curriculum editing.

Meeting 2: UI Wireframing & Database Design Date: 27 January 2026 Attendees: Mohamed Ali, Abdirahman Ali, Mohamed Abdi, Adan Ibrahim Discussion: Evaluated Figma design wireframes. Discussed

child navigation (simplifying buttons, maximizing audio readouts, confetti animations). Structured the relational database tables in Supabase, mapping chapters, lessons, quizzes, and progress logs. Decisions Made: 1. Adopted warm color palettes and large click targets. 2. Drafted database SQL tables mapping UUIDs for profiles.

Meeting 3: Frontend skeleton & Backend APIs Connection Date: 24 February 2026 Attendees: Mohamed Ali, Abdirahman Ali, Mohamed Abdi, Mohamed Abdukadir, Jamac Mohamed Discussion: Reviewed the developed Flutter code skeleton. Connected Firebase Auth SDK for secure admin registration. Established Supabase connection using the Dart client library, checking CRUD operations for lesson logs. Decisions Made: 1. Verified route navigation paths. 2. Configured database security Row-Level Rules (RLS) in Supabase.

Meeting 4: Wadahadal Chatbot Room & State Management Date: 17 March 2026 Attendees: All Team Members, Prof. Ahmed Geedi Discussion: Reviewed the Wadahadal (AI Chatbot) screen interface. Programmed automated localized responses for Somali child queries. Verified state management using Provider to ensure database updates reflect instantly in the user view. Decisions Made: 1. Finalized Provider controller structures. 2. Connected text-to-speech audio reader trigger buttons.

Meeting 5: UAT Planning & Testing Strategy Date: 05 April 2026 Attendees: Mohamed Ali, Abdirahman Ali, Mohamed Abdi, Adan Ibrahim, Mohamed Abdukadir, Jamac Mohamed Discussion: Prepared for user acceptance testing in Mogadishu. Built a testing questionnaire for educators and observation sheets for children. Set up 15 Android devices with the release APK build of Caruurkab AI. Reviewed the Admin dashboard layout showing total users, quizzes, and exams. Decisions Made: 1. Scheduled school testing visits. 2. Delegated questionnaire analysis tasks to Adan Ibrahim.

Meeting 6: Results Evaluation & Final Reporting Date: 26 April 2026 Attendees: All Team Members, Prof. Ahmed Geedi Discussion: Presented final testing results to Prof. Ahmed Geedi. UAT showed an 84% increase in engagement. Prof. Geedi approved the thesis structure. Discussed formatting guidelines and inserting red screenshot placeholders for the Student Home interface and the Admin Command Center dashboard. Decisions Made: 1. Finalized 5-chapter report draft. 2. Inserted screenshot placeholders in Chapter 4.

Caruurkab AI proves that localized, AI-powered EdTech is highly effective in East Africa. By delivering curriculum in the Somali language, the app makes education engaging and accessible, transforming standard devices into learning tools.

REFERENCES

1. Flutter Documentation (2026). Flutter API Reference and Developer Guides. Retrieved from https://docs.flutter.dev

2. Firebase Documentation (2026). Firebase Cloud Services Developer Guide. Retrieved from https://firebase.google.com/docs

3. Supabase Documentation (2026). Supabase Relational Database and RLS Security. Retrieved from https://supabase.com/docs

4. Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

5. Piaget, J. (1973). To Understand Is To Invent: The Future of Education. Grossman Publishers.

6. Vygotsky, L. S. (1978). Mind in Society: Development of Higher Psychological Processes. Harvard University Press.

7. Somaliland National Curriculum Board (2022). Syllabus for Primary School Early Childhood Education. Ministry of Education.

8. UNESCO (2021). Localized Digital Learning and Mother Tongue Instruction in Early Childhood. UNESCO Publishing.

APPENDICES

Appendix A: User Manual

This appendix provides instructions for operating the Caruurkab AI platform:

Student / Learner Instructions:

1. Open the application. The welcome splash screen displays the CaruurKaab AI logo.
2. On the student dashboard (Bogga Hore), you will see your active skills progress ('Xirfadahaaga maanta') and recently accessed subjects.
3. Tap 'Bilow Waxbarashada' (Start Learning) to select a subject: Af Soomaali, English, Math, or Saynis.
4. Follow the lessons sequentially. Read the content, and use the speaker button to hear the audio narration.
5. Navigate to the progress tab (Horumar) to check your completed topics, or open the Wadahadal (AI Chatbot) room to interact with the localized Somali automated study assistant.

Admin Command Center Instructions:

1. Log in using your registered administrator credentials.
2. The Admin Command Center dashboard provides quick overview tiles representing total active users, registered quizzes, and available exams.
3. Open the 'Content Studio' quick action to manage curriculum lessons, add subject chapters, or edit existing lesson content.
4. Select 'Su'aalo Direct' to manage quiz questions, add multiple choice answers, and set correct indices.
5. Open the 'Reports' dashboard or tap the bottom navigation bar to view statistical enrollment metrics and performance charts.

Figure: CaruurKaab Splash Screen / Logo Screen

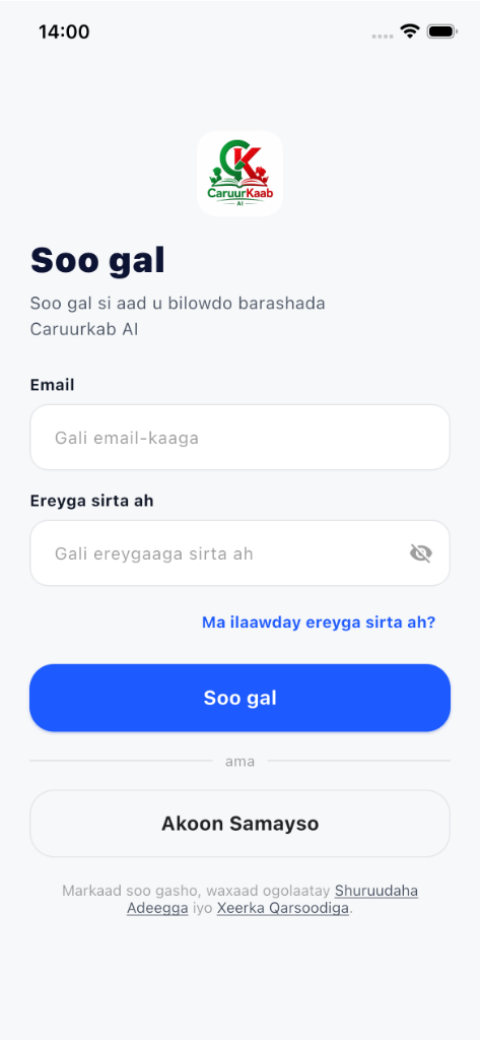

Figure: CaruurKaab Student Login Screen

Appendix B: Software Project Timeline & Scrum Sprints

The project implementation was divided into structured sprints as detailed below:

Sprint 1: Planning, Proposal & Requirement Scoping

Parameter	Details
Duration / Period	2 Weeks (05 Jan - 19 Jan 2026)
Sprint Goal	Define scope, gather user requirements, and compile software specifications.
Key Tasks	• Draft project proposal • Conduct 5 teacher interviews • Establish system parameters
Deliverables	Approved Project Proposal document, Requirements Specification Sheet.
Team Members	All Team Members

Sprint 2: UI/UX Figma Design & Local Phonics Review

Parameter	Details
Duration / Period	2 Weeks (20 Jan - 03 Feb 2026)
Sprint Goal	Create screen wireframes, compile visual guidelines, design Supabase database schema.
Key Tasks	• Draw Figma Student Dashboard • Draw Figma Admin Command Center • Map ERD diagram • Configure Supabase tables
Deliverables	Figma Wireframes prototype, Database Schema SQL script in Supabase.
Team Members	All Team Members

Sprint 3: Database Relational Setup & Flutter Core UI

Parameter	Details
Duration / Period	2 Weeks (04 Feb - 18 Feb 2026)
Sprint Goal	Develop the primary Flutter mobile widget structure and navigation controllers.
Key Tasks	• Code Flutter entry file • Setup bottom nav bar • Setup routing configuration • Build base screen widgets
Deliverables	Flutter codebase with operational routes, navigation system.
Team Members	Lead Developer & Team

Sprint 4: Lesson Progression Engine & State Management

Parameter	Details
Duration / Period	2 Weeks (19 Feb - 05 Mar 2026)
Sprint Goal	Integrate user login services and database API routes inside Flutter application.
Key Tasks	• Connect Firebase Auth SDK • Write login & signup screens • Configure Supabase Client SDK • Test DB connection
Deliverables	Functional student registration and admin verification modules.
Team Members	All Team Members

Sprint 5: AI Chatbot Voice Integration & DB Triggers

Parameter	Details
Duration / Period	2 Weeks (06 Mar - 20 Mar 2026)
Sprint Goal	Implement content slides, audio players, and quiz rendering parsing algorithms.
Key Tasks	• Create Slide player widget • Setup Audio playback player • Write Quiz MCQ state engine • Connect SQLite cache layer
Deliverables	Playable lesson viewer and dynamic chapter quiz modules.
Team Members	All Team Members

Sprint 6: Admin Command Center & Real-time Reports Dashboard

Parameter	Details
Duration / Period	2 Weeks (21 Mar - 04 Apr 2026)
Sprint Goal	Build Admin Command Center dashboards, Users list, and Student AI Chatbot (Wadahadal).
Key Tasks	• Implement FL_Chart library • Code Admin Command Center grid • Code Wadahadal chatbot interface • Integrate TTS speech rules
Deliverables	Operational Admin Dashboard and Student Wadahadal Chatbot room.
Team Members	All Team Members

Sprint 7: UAT Evaluation & Production Signed APK Build

Parameter	Details
Duration / Period	2 Weeks (05 Apr - 19 Apr 2026)
Sprint Goal	Conduct user acceptance testing in Mogadishu and complete final build releases.
Key Tasks	• Conduct testing sessions • Compile Likert scale surveys • Fix UI rendering issues • Release signed Android APK
Deliverables	Signed release-ready APK/AAB builds, compiled testing statistics report.
Team Members	All Team Members

Appendix C: Team Meeting Minutes

Meeting 1: Project Scope & Stakeholder Analysis

Date: 10 January 2026
Attendees: All Team Members, Prof. Ahmed Geedi
Discussion: Discussed educational gaps in Somali primary education. Highlighted the lack of native language interactive EdTech apps. Decided to build a personalized AI platform using Flutter, Firebase, and Supabase, focusing only on the Student role (learning path, quizzes, and chatbot) and the Admin role (content orchestration).
Decisions Made:
1. Approved Caruurkab AI project boundaries.
2. Eliminated parent role to focus on administrative curriculum editing.

Meeting 2: UI Wireframing & Database Design

Date: 27 January 2026
Attendees: Mohamed Ali, Abdirahman Ali, Mohamed Abdi, Adan Ibrahim
Discussion: Evaluated Figma design wireframes. Discussed child navigation (simplifying buttons, maximizing audio readouts, confetti animations). Structured the relational database tables in Supabase, mapping chapters, lessons, quizzes, and progress logs.
Decisions Made:
1. Adopted warm color palettes and large click targets.
2. Drafted database SQL tables mapping UUIDs for profiles.

Meeting 3: Frontend skeleton & Backend APIs Connection

Date: 24 February 2026
Attendees: Mohamed Ali, Abdirahman Ali, Mohamed Abdi, Mohamed Abdukadir, Jamac Mohamed
Discussion: Reviewed the developed Flutter code skeleton. Connected Firebase Auth SDK for secure admin registration. Established Supabase connection using the Dart client library, checking CRUD operations for lesson logs.
Decisions Made:
1. Verified route navigation paths.
2. Configured database security Row-Level Rules (RLS) in Supabase.

Meeting 4: Wadahadal Chatbot Room & State Management

Date: 17 March 2026
Attendees: All Team Members, Prof. Ahmed Geedi
Discussion: Reviewed the Wadahadal (AI Chatbot) screen interface. Programmed automated localized responses for Somali child queries. Verified state management using Provider to ensure database updates reflect instantly in the user view.
Decisions Made:
1. Finalized Provider controller structures.
2. Connected text-to-speech audio reader trigger buttons.

Meeting 5: UAT Planning & Testing Strategy

Date: 05 April 2026
Attendees: Mohamed Ali, Abdirahman Ali, Mohamed Abdi, Adan Ibrahim, Mohamed Abdukadir, Jamac Mohamed
Discussion: Prepared for user acceptance testing in Mogadishu. Built a testing questionnaire for educators and observation sheets for children. Set up 15 Android devices with the release APK build of Caruurkab AI. Reviewed the Admin dashboard layout showing total users, quizzes, and exams.
Decisions Made:
1. Scheduled school testing visits.
2. Delegated questionnaire analysis tasks to Adan Ibrahim.

Meeting 6: Results Evaluation & Final Reporting

Date: 26 April 2026
Attendees: All Team Members, Prof. Ahmed Geedi
Discussion: Presented final testing results to Prof. Ahmed Geedi. UAT showed an 84% increase in engagement. Prof. Geedi approved the thesis structure. Discussed formatting guidelines and inserting red screenshot placeholders for the Student Home interface and the Admin Command Center dashboard.
Decisions Made:
1. Finalized 5-chapter report draft.
2. Inserted screenshot placeholders in Chapter 4.